

IDEA
SUBMISSION

PARALLAX

Generative-AI Analysis & Risk
Scoring of Fraudulent APKs

SUBMITTED BY

Team TroN

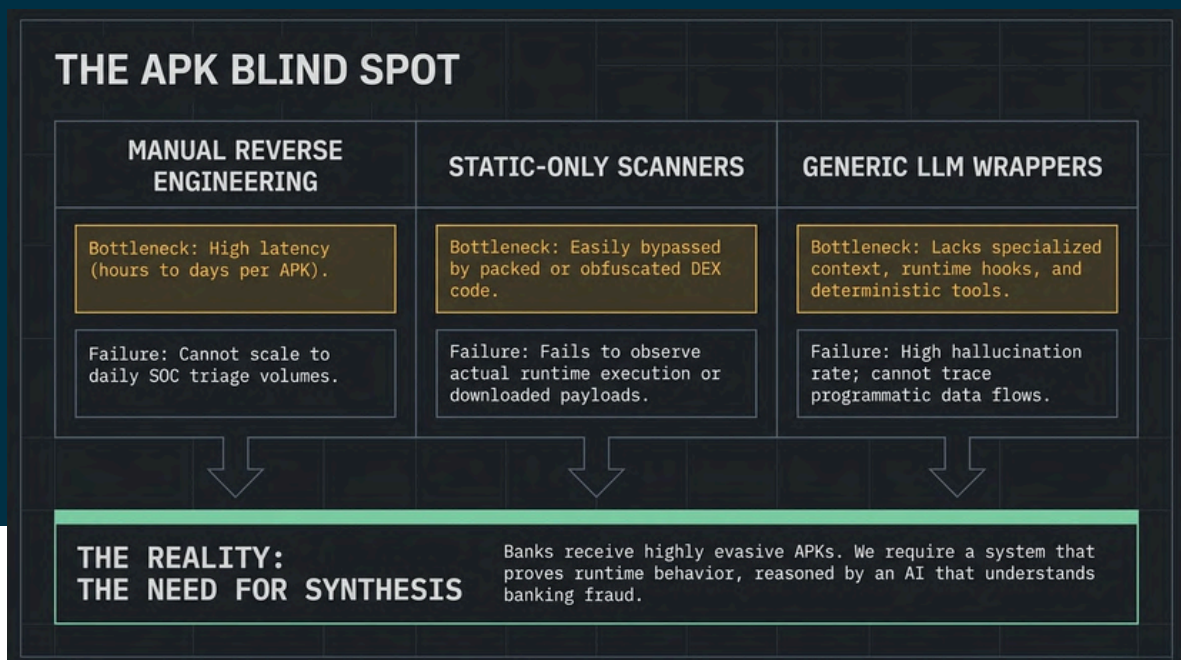
MEMBERS:

Tejesh Kunapareddy
Sahil Ahammad Shaik
Kanika Katare

ESCALATING BANKING FRAUD THREATS

₹22,845 Cr

Reported cyber-fraud losses in India in 2024 (I4C) — the threat PARALLAX is built to fight



Fraudsters push malicious APKs through WhatsApp, SMS, and phishing links, impersonating bank and government apps to steal credentials and intercept OTPs. The scale is systemic: India's I4C recorded about Rs 22,845 crore in reported cyber-fraud losses in 2024 across roughly 1.9 million complaints — nearly a three-fold rise year on year. Families such as SOVA, Drinik, Cerberus and TeaBot have hit customers of Indian banks; Drinik alone spanned 18 banks, and the SOVA campaign drew advisories from banks including Bank of India. Manual analysis cannot keep pace, and signatures fail against obfuscation, accessibility-gated payloads, and hourly repackaging.

EXECUTIVE SUMMARY

PARALLAX treats every submitted APK as a case to investigate, not a file to scan. A classical mobile-malware analysis stack (decompilation, taint analysis, an instrumented Android emulator) feeds a multi-agent generative-AI reasoning layer that interprets the evidence, attributes the malware family, and produces an explainable risk verdict. Every analysis also enriches a shared knowledge graph, so detection compounds with volume rather than resetting on each sample.

PARALLAX

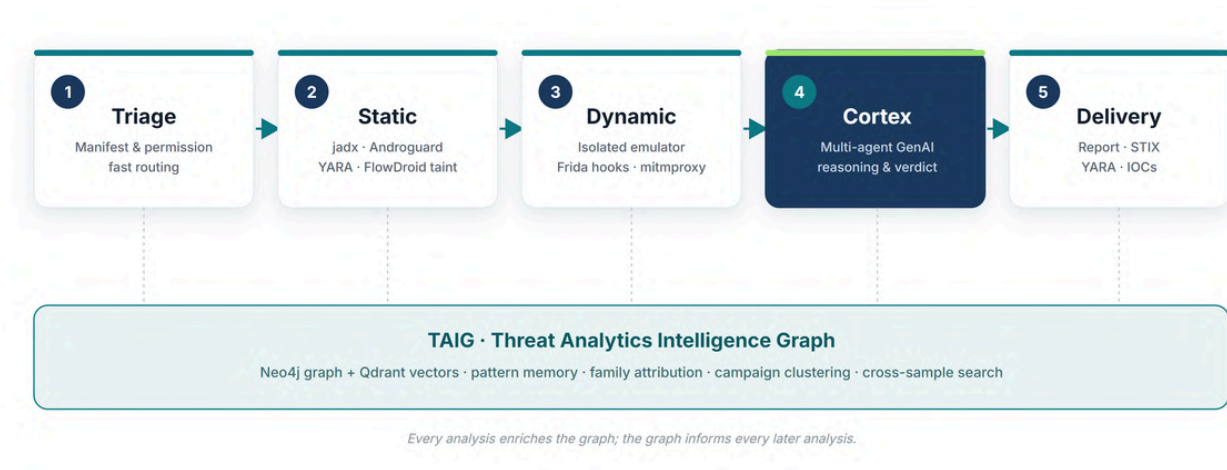
Persistent APK Risk Analysis via
Lateral LLM-Augmented eXpertise

Key Highlights

- Multi-agent GenAI system for end-to-end APK analysis
- Deterministic, auditable risk score with raw evidence links
- Static + dynamic analysis, not just signature matching
- On-premise deployment for regulated banking environments
- Outputs PDF reports, STIX, YARA, IOCs, and SIEM/SOAR webhooks

A FIVE-STAGE AUTOMATED PROCESS

Streamlining APK Fraud Detection with Generative AI

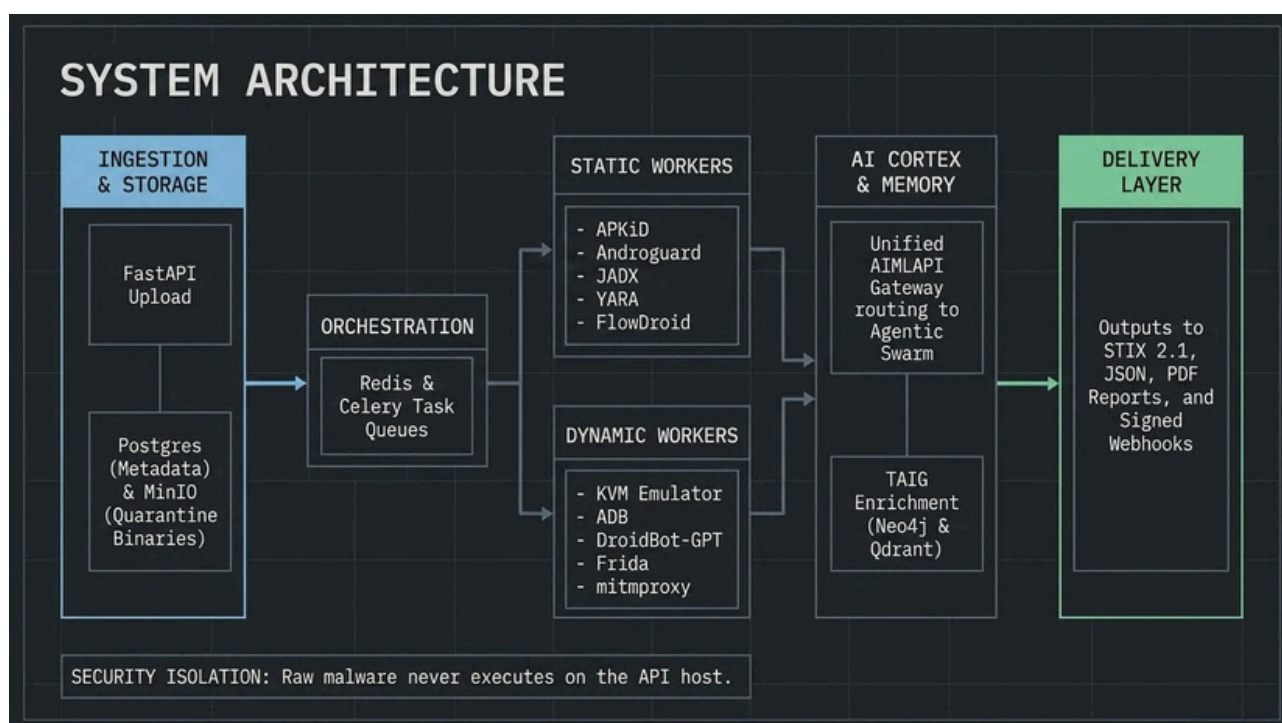


PARALLAX runs five stages. Triage performs a fast manifest and permission review to prioritise the sample. Static analysis decompiles the APK with jadx and Androguard, runs YARA pattern matching, FlowDroid source-to-sink taint analysis, and packer detection with APKiD.

Dynamic analysis executes the APK in an isolated, instrumented Android emulator with Frida runtime hooks and mitmproxy network capture. The Cortex reasons over the combined evidence, and Delivery emits a report plus machine-readable STIX, YARA and IOCs. A cross-cutting knowledge graph lets every analysis sharpen the next.

TECHNOLOGY STACK & DELIVERABLES

Open-source foundations, with a fully on-premise option



Static: jadx, Androguard, YARA, APKiD, FlowDroid. Dynamic: Frida, mitmproxy, an Android emulator. GenAI: a tiered model layer that can run fully local via Ollama (phi-3, mistral, qwen2.5-coder, llava), or use a cloud gateway when configured. Knowledge: Neo4j and Qdrant.

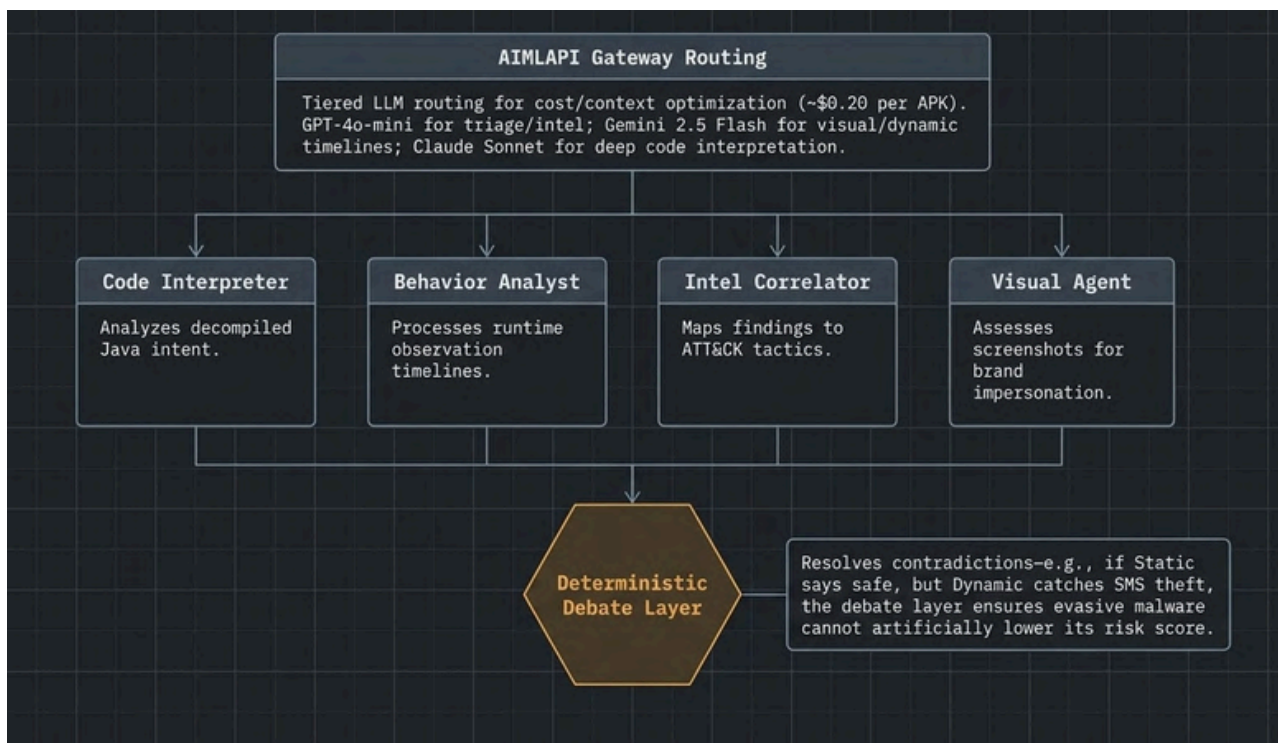
Platform: FastAPI, Celery, Redis, PostgreSQL, MinIO, with Prometheus and Grafana for observability. Deliverables per analysis: an evidence-first PDF report, a risk score with full breakdown, a STIX 2.1 bundle, an auto-generated YARA rule, signed webhooks to SIEM/SOAR, and a tamper-evident evidence bundle.

REASONING CORTEX

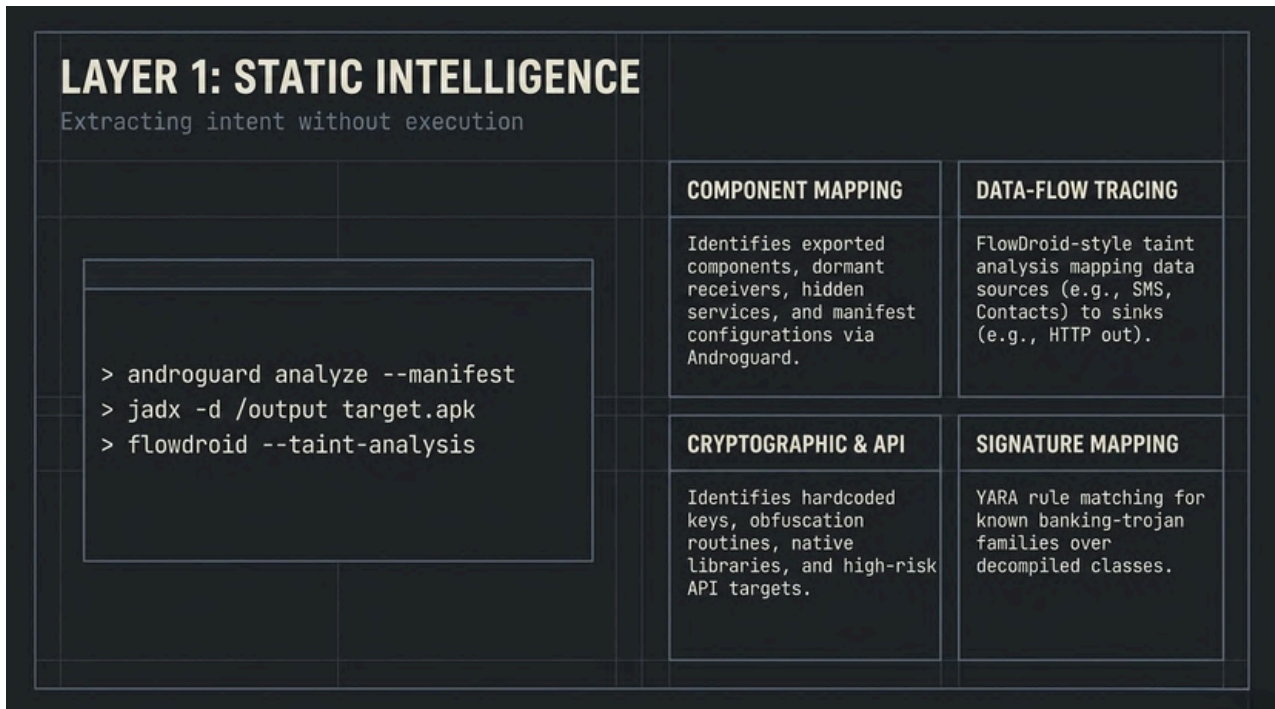
Seven specialist agents, tiered by task

The Cortex is a multi-agent system. Three agents read evidence in parallel: Code Interpreter classifies the intent of decompiled code; Behaviour Analyst reconstructs the runtime kill-chain; and Visual Intel detects overlay attacks and brand impersonation in screenshots.

Intel Correlator maps behaviour to MITRE ATT&CK and attributes the family via graph RAG; Debate surfaces static-versus-dynamic contradictions as signal. The Risk Scorer is deterministic — a weighted, auditable evidence score, not an LLM guess — and Synthesis writes the evidence-first verdict and report.

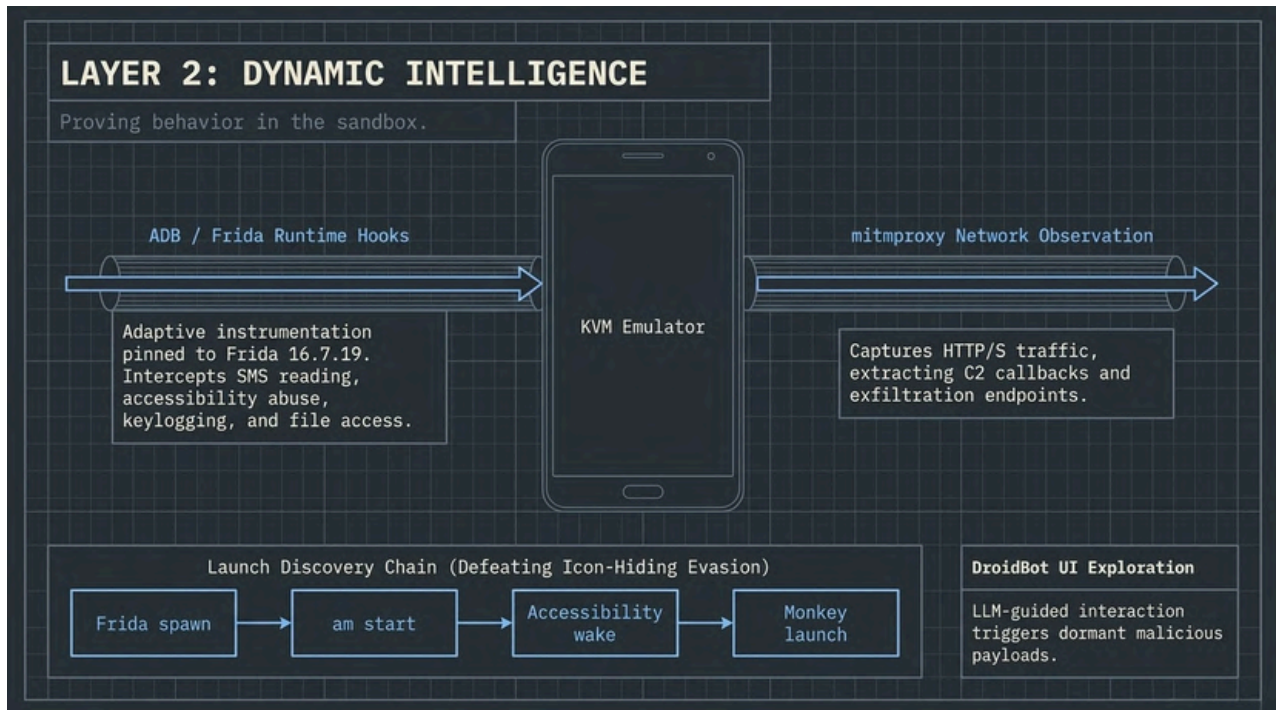


A TWO-LAYERED APPROACH



Static Intelligence builds the first evidence layer before the APK is executed. PARALLAX inspects manifest and certificate anomalies, permissions, exported components, decompiled code intent, suspicious APIs, strings, URLs, domains, native libraries, packer and obfuscation signals, and FlowDroid-style source-to-sink flows. These findings establish the baseline risk score by identifying what the APK is capable of doing, which banking-fraud tactics it resembles, whether it matches known malware families, and what behavior should be verified later through dynamic runtime observation.

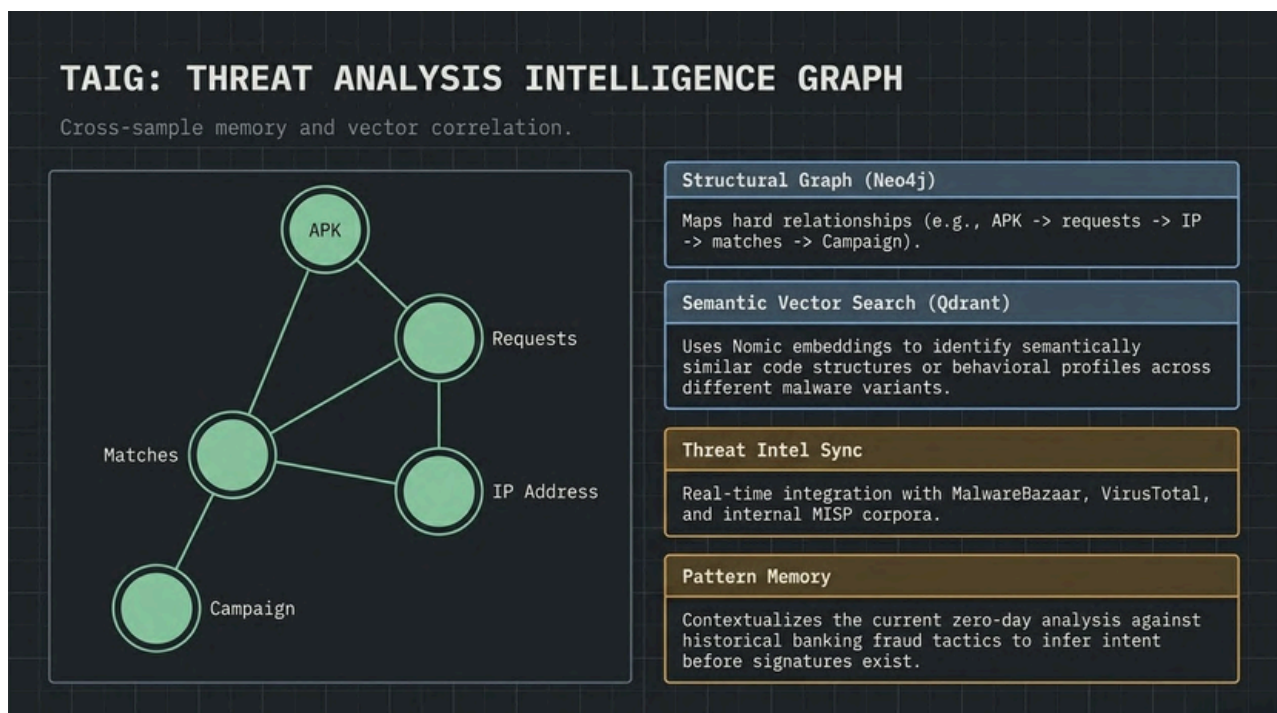
A TWO-LAYERED APPROACH



Dynamic Intelligence verifies what the APK actually does when executed. PARALLAX runs the sample inside an isolated Android emulator, installs and launches it through ADB, explores user flows with DroidBot-GPT, and observes runtime behavior using Frida hooks and mitmproxy network capture. This layer records live evidence such as API calls, file access, command execution, SMS behavior, overlay abuse, accessibility-service misuse, network callbacks, C2 communication, and anti-analysis signals. It confirms or challenges the static findings, turning suspected capabilities into observed behavior that strengthens the final risk score and analyst verdict.

ILLUMINATING THE THREAT LANDSCAPE

Every analysis enriches a Threat Analytics Intelligence Graph (Neo4j for structure, Qdrant for semantic vectors). An APK is linked to its permissions, domains and IPs, MITRE ATT&CK techniques, malware family and campaign — enabling cross-sample threat hunting and campaign detection. This turns per-file analysis into an intelligence product that sharpens with every new sample.



DEPLOYMENT & GOVERNANCE

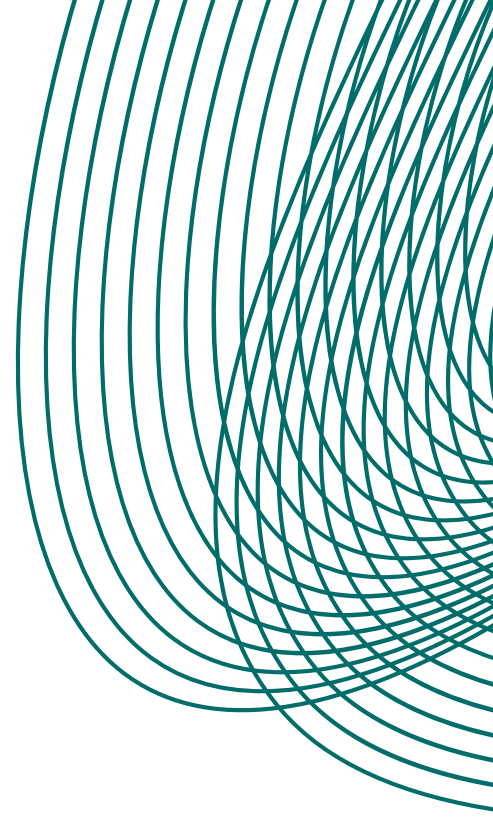
Banking Infrastructure Compliance

PARALLAX runs as containerised services and can operate fully on-premise or air-gapped on local models, so no APK or derived data needs to leave the bank. Samples are handled as live malware: quarantined in object storage, never executed on the API host, and run only inside isolated emulators.

Outbound sandbox traffic is confined to the capture proxy, every action is logged immutably, and each verdict ships with a signed, tamper-evident evidence bundle. The design aligns with the RBI Cyber Security Framework and data-localisation expectations for regulated financial institutions.

The design answers the three properties that defeat signature-based defenses: heavy runtime obfuscation, malicious surfaces hidden behind accessibility and notification services, and hourly polymorphic repackaging. The verdict is produced by a deterministic, auditable scoring model over evidence items; the language model writes the narrative, it does not decide the score.

LET'S SECURE BANKING



We also moved beyond planning and started turning PARALLAX into a working prototype. I began testing the framework with real APK samples sourced from MalwareBazaar, using them to validate static analysis, dynamic sandboxing, agent-based reasoning, and APK-to-verdict generation. Along the way, I am exploring new GenAI techniques and technologies such as agent chatrooms, LLM-assisted triage, specialist security agents, evidence correlation, and automated security reporting. The implementation progress, experiments, and current framework are recorded in my GitHub repository: <https://github.com/arjun7n9s/Parallax>